RFC-197 - Android Addendum

Android Libraries

Attending to the RFC-197 requirements and making the solution aligned with the development standards future, the Android Library will use the following libraries in its creation. First, let's make a quick note about the *Gradle files*:

Google is leaving explicit in their documentation that in the future Kotlin DSL (KTS) will be preferred over Groovy for writing Gradle scripts for Android projects (Migrate your build configuration from Groovy to KTS | Android Developers).

Kotlin is more readable, has better compile-time checking, and offers good integration in Android Studio's code editor when compared to Groovy.

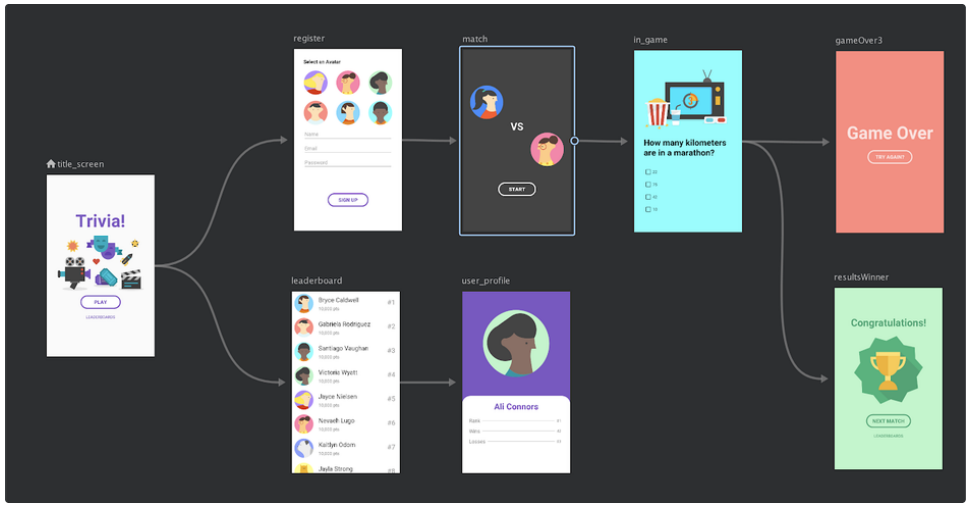
That being said, we are going to use Kotlin DSL from scratch in the Library.

Now, let's start talking about Android libraries!

Navigating

Jetpack Navigation (Get started with the Navigation component | Android Developers) was created by Google to simplify the navigation implementation.

Is safe, and easy to assist you in visualizing your app's navigation flow.

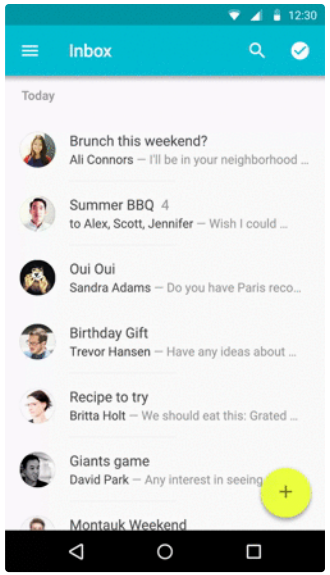


User Interface

To manipulate the UI elements, we need to get the element reference and then take some action. **Jetpack ViewBinding** (View Binding | Android Developers) is the choice for handling the task. It's a feature created by Google that allows you to more easily write code that interacts with views.

On the other hand, when we need to deal with List views the “default” and better solution is to use the RecyclerView library (Create dynamic lists with RecyclerView | Android Developers).

Sometimes, as users, we want to see the latest data without the need to jump to another page and go back for it. The most common approach for it is to *swipe down to refresh*.



In Android, the common "pull to refresh" UX concept is not built into a List component. To add this behavior we need to use the **SwipeRefreshLayout** (Swiperefreshlayout | Desenvolvedores Android | Android Developers) library. It uses a vertical swipe gesture to update the content of views.

Model-View-ViewModel (MVVM)

The MVVM pattern suggests separating the data presentation logic (Views or UI) from the main business logic part of the application.

Our business logic is going to be implemented in the ViewModel class, and for that, we are going to use the **Jetpack ViewModel** (ViewModel overview | Android Developers).

This library helps us to work with ViewModels that store and manage UI-related data in a lifecycle-conscious way.

To get the latest value from the ViewModel and display it on the UI, we are going to implement the **StateFlow** (StateFlow and SharedFlow | Android Developers) library.

A StateFlow is an observable data holder which gives always the last value of the pipeline. Is really useful for representing a view state.

Network

The most common way to consume/request data from the *backend* is by making HTTP calls.

Thinking on it, on Android the best and almost the *default* way for dealing with HTTP calls is to use both, OkHttp (OkHttp) and Retrofit (Retrofit).

The **OkHttp** library is a pure HTTP client responsible for any low-level network operations, such as caching, requests, and response manipulation.

In other words, OkHttp helps us to request and get data from the backend. We can also use it for creating Interceptors and add it as a client to the Retrofit instance.

Under the hood, **Retrofit** is a high-level REST abstraction built on top of OkHttp that allows working with threads (asynchronous calls).

For dealing with async tasks, we are going to use **Flow** and Retrofit together.

Async Tasks

Here, our champion is **Flow** (Kotlin Flow).

Flows are built on top of coroutines and can provide multiple values. A flow is conceptually a *stream of data* that can be computed asynchronously and emit values of the same type.

Different from *StateFlow* (**hot flow**) mentioned in the MVVM section, *Flows* (**cold flow**) don't hold values and just emit a value when there are collectors/observers/consumers.

Dependency Injection (DI)

The dependency injection pattern helps us to decouple the code and make the application more testable.

On Android, the newest, most friendly, robust, stable, and created by Google dependency injection system is **Hilt** (Dependency injection with Hilt | Android Developers).

Storing Data

We are not going to store and handle non-trivial amounts of structured data locally. Keeping that in mind, all we need is to use **Jetpack DataStore** (App Architecture: Data Layer - DataStore - Android Developers).

DataStore (created by Google) is a data storage solution that allows you to store key-value pairs or typed objects with protocol buffers. DataStore uses Kotlin coroutines and Flow to store data asynchronously, consistently, and transactionally.

Testing

We are going to use **JUnit** which is a Java library used to write tests and the **Mockito** library which enables writing tests using the mocking approach (Tests | Android Developers).

Logging

To have better logs, the choice here is **Timber** (GitHub - JakeWharton/timber) is an API for Android's Log class. It basically enhances the logs from Android.

Stripe SDK

And last but not the least, we are going to use **Stripe SDK** (GitHub - stripe/stripe-android: Stripe Android SDK) for Financial Connections support and Identity Verification.